

Survey on Sparse Matrix-Vector Multiplication

r13943052 黃秉璿
陳靖雯

Contents

- Introduction of Sparse Matrix-Vector Multiplication (SpMV)
- Formats for SpMV
- Structure of Streaming Data Engine for SpMV
- Performance Model
- Algorithms
- Environment Setup and Experiment Results
- HiSparse: High-Performance Sparse Linear Algebra on HBM-Equipped FPGAs

Introduction of SpMV

- What SpMV is
 - Computes $y = A \cdot x$, where A is sparse ($\ll n^2$ non-zeros)
 - Memory-bound: arithmetic intensity is typically $\leq 1 \text{ FLOP} / 12 \text{ B}$
 - Core of graph analytics, iterative solvers, recommendation engines
- Why it is hard to accelerate
 - Irregular memory access – non-zeros arrive with random column indices $\rightarrow$ cache/TLB misses on CPUs/GPUs
 - Bandwidth limited – moving the values & indices dominates run time; extra FLOPs don't help if DDR is the bottleneck
 - Load imbalance – row lengths vary widely; SIMD lanes idle on short rows

Formats for SpMV

- Various type of formats

- CSR (Compressed Sparse Row)
- COO (Coordinate list)
- ELLPACK
- BCSR (Block CSR)

$$\begin{array}{c}
 m=5 \\
 n=4 \\
 \left[\begin{array}{ccccc}
 2 & 0 & 0 & 1 & 0 \\
 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 1 & 4 \\
 0 & 2 & 0 & 1 & 0
 \end{array} \right] \\
 \text{nnz}=6
 \end{array}$$

(a)

$A = \{ (0, 0, 2), (0, 3, 1), (2, 3, 1), (2, 4, 4), (3, 1, 2), (3, 3, 1) \}$

row_index $\begin{bmatrix} 0 & 0 & 2 & 2 & 3 & 3 \end{bmatrix}$
 col_index $\begin{bmatrix} 0 & 3 & 3 & 4 & 1 & 3 \end{bmatrix}$
 value $\begin{bmatrix} 2 & 1 & 1 & 4 & 2 & 1 \end{bmatrix}$

(b)

row_index $\begin{bmatrix} 0 & 2 & 2 & 4 & 6 \end{bmatrix}$
 value $\begin{bmatrix} 2 & 1 & 1 & 4 & 2 & 1 \end{bmatrix}$
 col_index $\begin{bmatrix} 0 & 3 & 3 & 4 & 1 & 3 \end{bmatrix}$

(c)

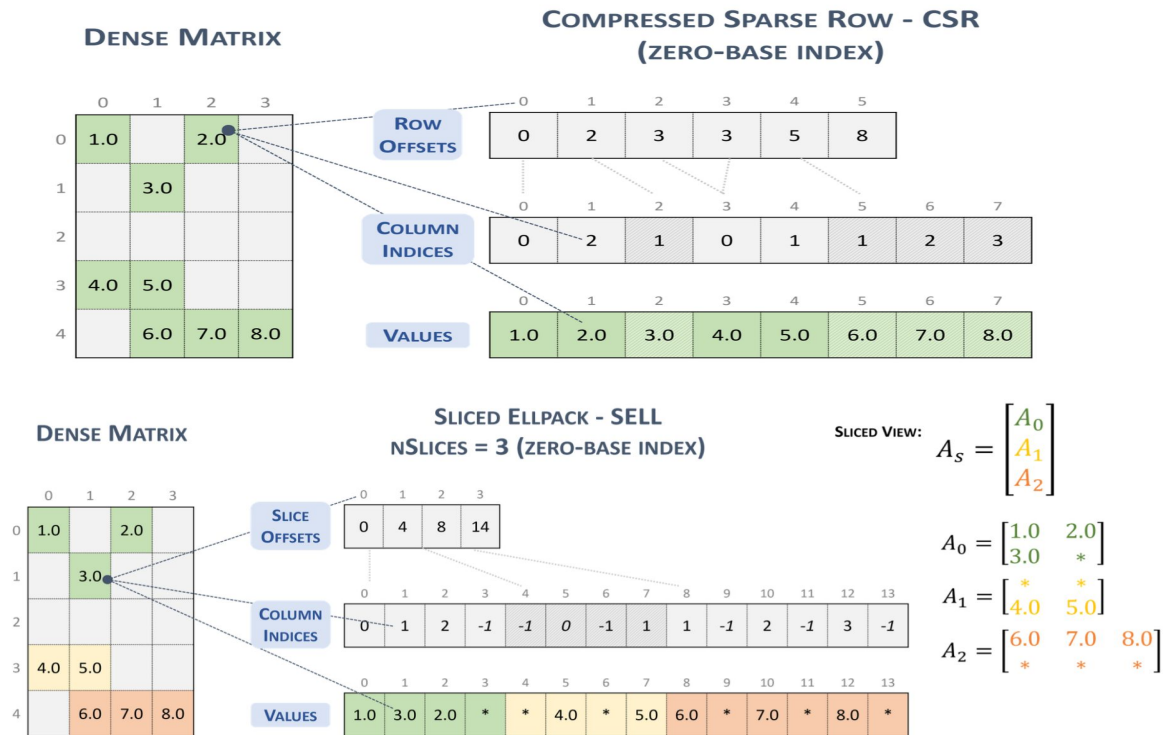
- Choosing a format

- CPU / FPGA streaming → CSR / BCSR (row-major access, dataflow friendly)
- GPU kernels → ELL, HYB, JDS (warp-coalesced, uniform row length)
- Stencil / banded PDE → DIA (few diagonals, minimal index storage)
- Dynamic insertion → COO (simple append—later convert to CSR/ELL)

- Rule of thumb: balance indexing load vs. padding waste.

- High hardware parallelism benefits from regular row length (ELL), while small FPGAs prefer minimal index traffic (CSR/DIA)

CSR versus ELLPACK



Structure of Streaming Data Engine for SpMV

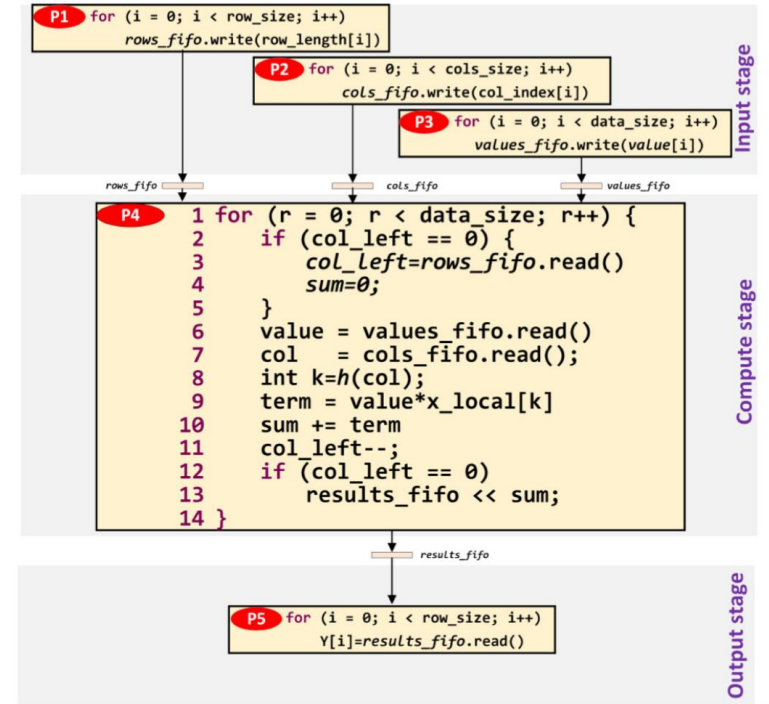
```
void spvm_kernel(
    DATA_TYPE *values,
    u32 *cols,
    u32 *rows,
    DATA_TYPE *x_local,
    DATA_TYPE *y,
    u32 row_size,
    u32 col_size,
    u32 data_size
) {
    #pragma HLS DATAFLOW

    // Declare local streams/FIFOs
    hls::stream<u32> rows_fifo("rows_fifo");
    hls::stream<DATA_TYPE> values_fifo("values_fifo");
    hls::stream<u32> cols_fifo("cols_fifo");
    hls::stream<DATA_TYPE> results_fifo("results_fifo");

    // Optionally set a custom depth to avoid deadlock warnings:
    #pragma HLS STREAM variable=rows_fifo depth=64
    #pragma HLS STREAM variable=values_fifo depth=64
    #pragma HLS STREAM variable=cols_fifo depth=64
    #pragma HLS STREAM variable=results_fifo depth=64

    // (1) Read from memory into streams
    read_data(values, cols, rows,
              values_fifo,
              cols_fifo,
              rows_fifo,
              row_size,
              data_size);

    // (2) Do the actual spmv multiply-accumulate
    compute(values_fifo, cols_fifo, rows_fifo,
            results_fifo,
            x_local,
            row_size,
            data_size);
}
```



Structure of Streaming Data Engine for SpMV

```
// -----  
// Function to read from global memory (only one  
// -----  
static void read_data(  
    DATA_TYPE *values,  
    u32 *cols,  
    u32 *rows,  
    hls::stream<DATA_TYPE> &values_fifo,  
    hls::stream<u32> &cols_fifo,  
    hls::stream<u32> &rows_fifo,  
    u32 row_size,  
    u32 data_size  
) {  
    // Read rows  
    for (u32 i = 0; i < row_size; i++) {  
        #pragma HLS PIPELINE II=1  
        rows_fifo << rows[i];  
    }  
  
    // Read values and cols  
    for (u32 i = 0; i < data_size; i++) {  
        #pragma HLS PIPELINE II=1  
        values_fifo << values[i];  
        cols_fifo << cols[i];  
    }  
}
```

```
// -----  
// Function that does the SPMV multiply-accumulate  
// -----  
static void compute(  
    hls::stream<DATA_TYPE> &values_fifo,  
    hls::stream<u32> &cols_fifo,  
    hls::stream<u32> &rows_fifo,  
    hls::stream<DATA_TYPE> &results_fifo,  
    DATA_TYPE *x_local,  
    u32 row_size,  
    u32 data_size  
) {  
    u32 col_left = 0;  
    DATA_TYPE sum = 0;  
  
    // For each nonzero element  
    for (u32 r = 0; r < data_size; r++) {  
        #pragma HLS PIPELINE II=1  
        if (col_left == 0) {  
            // read how many columns in the next row  
            col_left = rows_fifo.read();  
            sum = 0;  
        }  
  
        DATA_TYPE value = values_fifo.read();  
        u32 col = cols_fifo.read();  
        sum += value * x_local[col];  
  
        col_left--;  
        if (col_left == 0) {  
            // end of this row => push sum out  
            results_fifo << sum;  
        }  
    }  
}
```

Structure of Streaming Data Engine for SpMV

```
int spmv_accel(
    DATA_TYPE    values[DATA_LENGTH],
    u32           cols[DATA_LENGTH],
    u32           rows[ROWS],
    DATA_TYPE    x[COLS],
    DATA_TYPE    y[ROWS],

    u32           row_size,
    u32           col_size,
    u32           data_size
) {
    DATA_TYPE    x_local[MAX_COL_SIZE];

    for (u32 i = 0; i < col_size; i++) {
#pragma HLS PIPELINE
        x_local[i] = *(x+i);
    }

    spvm_kernel(values, cols, rows, x_local, y, row_size, col_size, data_size);

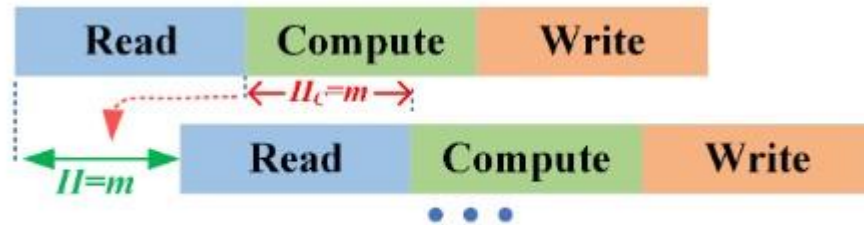
    return 0;
}

// -----
// Function to write results back to global memory (only one process w
// -----
static void write_data(
    hls::stream<DATA_TYPE> &results_fifo,
    DATA_TYPE *y,
    u32 row_size
) {
    for (u32 i = 0; i < row_size; i++) {
#pragma HLS PIPELINE II=1
        y[i] = results_fifo.read();
    }
}
```


Performance Model & input matrix generation

- T = time required by algorithm + platform overhead
- $t^{\text{alg}} = (m + \text{nnz}) / f$
- $t^{\text{plat}} = (l - 1) \text{nnz} / f$
- $T = (m + l * \text{nnz}) / f$

$$T = t^{\text{alg}} + t^{\text{plat}} = \left(t_{\text{ideal}}^{\text{alg}} + t_{\text{over}}^{\text{alg}} \right) + \left(t_{\text{lib}}^{\text{plat}} + t_{\text{hard}}^{\text{plat}} \right).$$



(c) Pipelined streaming computation with $l > 1$

```
%MatrixMarket mat
% A tiny 5x5 sparse
5 5 10
1 1 1.5
1 3 2.3
2 2 3.7
2 4 1.2
3 1 0.8
3 3 4.5
4 4 5.6
4 5 1.9
5 2 0.7
5 5 3.2
```

Algorithm 1: Naïve Stream Computing

- Problems for running csim
 - 1. Dataflow unavailable due to the potential deadlock
 - 2. SDS derepated
- Problems for running cosim
 - 1. Segmentation Fault (buffer overflow)
- Solutions:
 - modulize the functions into load, compute and write
 - use cstdlib instead of sds_lib
 - When it comes to segmentation fault, situation becomes a bit tricky

```
//-----  
// Memory macros for SDSoc or fallback for plain HLS  
//-----  
#ifdef __SDSCC__ // SDSoc / Vitis embedded  
#include "sds_lib.h"  
#else  
#include <cstdlib>  
#define sds_alloc_non_cacheable(sz) malloc(sz)  
#define sds_free(ptr) free(ptr)  
#endif
```

Report time : Tue 10 Jun 2025 08:00:42 PM CST.
Solution : solution1.
Simulation tool : xsim.

RTL	Status	Latency(Clock Cycles)			Interval(Clock Cycles)			Total Execution Time (Clock Cycles)
		min	avg	max	min	avg	max	
VHDL	NAI	NAI	NAI	NAI	NAI	NAI	NAI	NAI
Verilog	Pass	509	519	540	489	504	520	1510

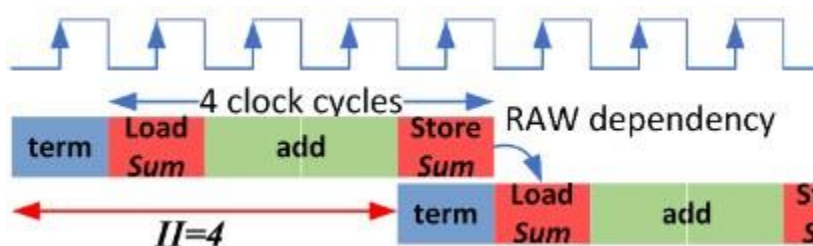
Algorithm 1: Direct CSR SpMV

- Use software code as hls code
- The inner loop bounds are variables

```
1 void SpMV_Ref(int n, float *value, int *col_index, int *  
   row_index, float *x, float *y) {  
2   int rowStart = 0, rowEnd = n;  
3  
4   for (int i = rowStart; i < rowEnd; ++i) {  
5     float y0 = 0.0;  
6     for (int j=row_index[i]; j<row_index[i+1]; j++) {  
7       int k = col_index[j];  
8       y0 += value[j] * x[k];  
9     }  
10    y[i] = y0;  
11  }  
12 }
```

Algorithm 2: Naïve Stream Computing

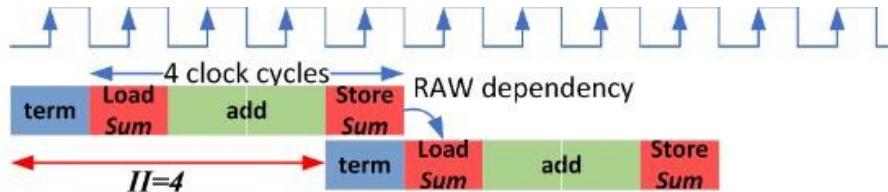
- data in rows_fifo need to be preprocessed to MCSR if input matrix is in CSR format
- Read after write hazard in sum and col_left -> II != 1
- Bug in the code: pure 0 row will cause error



```
// -----  
// Function that does the SPMV multiply-accumulate  
// -----  
static void compute(  
    hls::stream<DATA_TYPE> &values_fifo,  
    hls::stream<u32> &cols_fifo,  
    hls::stream<u32> &rows_fifo,  
    hls::stream<DATA_TYPE> &results_fifo,  
    DATA_TYPE *x_local,  
    u32 row_size,  
    u32 data_size  
) {  
    u32 col_left = 0;  
    DATA_TYPE sum = 0;  
  
    // For each nonzero element  
    for (u32 r = 0; r < data_size; r++) {  
        #pragma HLS PIPELINE II=1  
        if (col_left == 0) {  
            // read how many columns in the next row  
            col_left = rows_fifo.read();  
            sum = 0;  
        }  
  
        DATA_TYPE value = values_fifo.read();  
        u32 col = cols_fifo.read();  
        sum += value * x_local[col];  
  
        col_left--;  
        if (col_left == 0) {  
            // end of this row => push sum out  
            results_fifo << sum;  
        }  
    }  
}
```

Algorithm 3: Fast Stream Computing

- Fast streaming : uroll the loop by II
(need to pad each row to be a multiple of II)
- Process II terms in one iteration $\sim II = 1$
(Cons: II times of MAC circuit)



(a) Naïve implementation



```

1  for (r=0; r<data_size; r +=  $II_{com}$ ) { //pipelined
2      if (col_left == 0) {
3          col_left=rows_fifo.read()
4          sum=0;
5      }
6      for (int i = 0; i <  $II_{com}$ ; i++) { //unrolled
7          value = values_fifo.read();
8          col = col_fifo.read();
9          int k = h(col);
10         y[i] = y0;
11         term[i] = value * x[k];
12     }
13     DATA_TYPE sum_tmp=0;
14     for (int i = 0; i <  $II_{com}$ ; i++) { //unrolled
15         sum_tmp += term[i];
16     }
17     sum += sum_tmp;
18     col_left -=  $II_{com}$ ;
19     if (col_left == 0) {
20         results_fifo << sum;
21     }
22 }
    
```

Algorithm 3: Fast Stream Computing

- What if uroll factor (UR) > II ?
 - The bottleneck will become fifo since only one element can be read/write in one cycle.
 - Using stream<vector> can solve the above issue, but large UR still cause timing violation or increas II. In conclusion, speeding up computation by increasing UR is not scalable

```
1 for (r=0; r<data_size; r+= IIcom) { //pipelined
2   if (col_left == 0) {
3     col_left=rows_fifo.read()
4     sum=0;
5   }
6   for (int i = 0; i < IIcom; i++) { //unrolled
7     value = values_fifo.read();
8     col = col_fifo.read();
9     int k = h(col);
10    y[i] = y0;
11    term[i] = value * x[k];
12  }
13  DATA_TYPE sum_tmp=0;
14  for (int i = 0; i < IIcom; i++) { //unrolled
15    sum_tmp += term[i];
16  }
17  sum += sum_tmp;
18  col_left-=IIcom;
19  if (col_left == 0) {
20    results_fifo << sum;
21  }
22 }
```

Reduced-Port Stream Computing

- Technique involved: combine the row and column into one array
- Cons: Time complexity = $O(\text{nnz} + m)$ if data is processed on chip

```
while (processed < PAD_TOTAL) {
#pragma HLS PIPELINE II=1
    if (row_fifo.empty() && col_left == 0) continue;
    if (col_fifo.size() < II || val_fifo.size() < II) continue;

    if (col_left == 0) {
        col_left = row_fifo.read();
        sum = 0;
    }

READ_II:
    for (int i = 0; i < II; ++i) {
#pragma HLS UNROLL
        DATA_TYPE v = val_fifo.read();
        u32 c = col_fifo.read();
        term[i] = v * x_local[c];
    }

    DATA_TYPE sum_tmp = 0;
REDUCE_II:
    for (int i = 0; i < II; ++i) {
#pragma HLS UNROLL
        sum_tmp += term[i];
    }
    sum += sum_tmp;
    col_left -= II;
    processed += II;

    if (col_left == 0) res_fifo << sum;
}
```

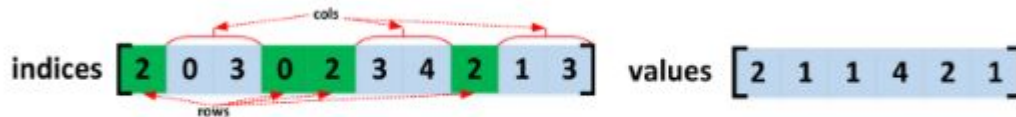


Fig. 7: Two-port streaming CSR

Multiport Stream Computing

- Suppose FPGA has P memory ports, each having B bits and value & indices have g and h bits
- The rows to be process at the same time $p \leq P * B / (g + h)$

Algorithm 4: Load Balancing

- eup : non-zero elements after padding
- Ideal partition : equally loaded to P computing unit
- Greedy partition : loaded rows to P computing units with same eup as equal as possible

Algorithm 1: Load balancing algorithm

Data: *no_part*: number of partition

Data: *eup*: number of total eup

Data: $R = \langle r_0, r_1, \dots, r_{N-1} \rangle$:

Result: $\langle P_0, P_1, P_{p-1} \rangle$:

```
1 ideal_part_size = eup/no_part;  
2  $P_0 = r_0$   
3  $j = 0$ ;  
4 for  $i \leftarrow 1$  to  $N - 1$  do  
5   if  $|P_j| + |r_i| < \textit{ideal\_part\_size}$  then  
6      $P_j = P_j + r_i$   
7   else  
8     if  $j + 1 < \textit{no\_part}$  then  
9        $j++$ ;  
10    end  
11     $P_j = P_j + r_i$   
12  end  
13 end
```

Experiment Setup and Results

(note: The interface of Case 2,3,4 are changed to `hls::stream`, MAX_SZ=20000)

Two Dataset:

1. Sparse Test Dataset (97.5% sparsity in 256 x 256 matrix, nnz = 1638)
2. Denser Test case (70% sparsity in 256 x 256 matrix, nnz = 19661)

T = 10ns

ll = 4 in case 2

Experiment Setup and Results

Case	1	2	3 (II=4)	3.1 (II=8)	4 (II=4, P=2)
Configuration	Initial (CSR)	Naive Streaming	Fast Streaming	Fast Streaming	Load Balancing + Fast Streaming
BRAM	8	10	10	14	74
Latency 1	15445	8359	2297	2552	1277
Latency 2	136532(X)	98574	20582	21112	10490
DSP48E	5	5	7	7	14
FF	3353	1202	3628	3830	7056
LUT	4462	1878	4140	4791	7967

Experiment Setup and Results

- Case 1 co-sim mismatched at last row in larger dataset
- Array as hls function parameter
 - Hard to meet dataflow requirement : need to bind each port to different memory port
 - Use ap_fifo as interface

```
void spmv_opt1(const int row_ptr[MAX_M],
               const int col_idx[MAX_SZ],
               const DATA_TYPE val[MAX_SZ],
               const DATA_TYPE x[MAX_N],
               DATA_TYPE y[MAX_M],
               int M, int N, int nnz)
{
#pragma HLS INTERFACE m_axi port=row_ptr depth=1024
#pragma HLS INTERFACE m_axi port=col_idx depth=50000
#pragma HLS INTERFACE m_axi port=val depth=50000
#pragma HLS INTERFACE m_axi port=x depth=1024
#pragma HLS INTERFACE m_axi port=y depth=1024
```

```
Compiling apatb_spmv_opt1.cpp
Compiling tb.cpp_pre.cpp.tb.cpp
Compiling spmv_balance_opt4.cpp_pre.cpp.tb.cpp
Compiling spmv_csr_opt1.cpp_pre.cpp.tb.cpp
Compiling apatb_spmv_opt1_ir.ll
Generating cosim.tv.exe
INFO: [COSIM 212-302] Starting C TB testing ...
[mismatch] row 255 gold=1356.13 hw=0
x FAIL - 1 mismatches.
ERROR: [COSIM 212-359] Aborting co-simulation: C T
ERROR: [COSIM 212-320] C TB testing failed, stop g
ERROR: [COSIM 212-5] *** C/RTL co-simulation file
ERROR: [COSIM 212-4] *** C/RTL co-simulation finis
```

Experiment Setup and Results

- Configurations:
- Challenges:
- Results:

	Numbers of cycles	Cycle time	Resource utilization

Hardware Emulation Result for simple SpMV

Timing Information (MHz)				
Compute Unit	Kernel Name	Module Name	Target Frequency	Estimated Frequency
spmv_accel_1	spmv_accel	spmv_accel_Pipeline_init_loop	300.300293	551.572021
spmv_accel_1	spmv_accel	spmv_accel_Pipeline_compute_loop	300.300293	265.111328
spmv_accel_1	spmv_accel	spmv_accel_Pipeline_wb_loop	300.300293	411.353363
spmv_accel_1	spmv_accel	spmv_accel	300.300293	265.111328

Latency Information			Start Interval	Best (cycles)	Avg (cycles)	Worst (cycles)	Best (absolute)	Avg (absolute)
Compute Unit	Kernel Name	Module Name						
spmv_accel_1	spmv_accel	spmv_accel_Pipeline_init_loop	1026	1026	1026	1026	3.417 us	3.417 us
spmv_accel_1	spmv_accel	spmv_accel_Pipeline_compute_loop	undef	undef	undef	undef	undef	undef
spmv_accel_1	spmv_accel	spmv_accel_Pipeline_wb_loop	1027	1027	1027	1027	3.420 us	3.420 us
spmv_accel_1	spmv_accel	spmv_accel	undef	undef	undef	undef	undef	undef

Area Information							
Compute Unit	Kernel Name	Module Name	FF	LUT	DSP	BRAM	URAM
spmv_accel_1	spmv_accel	spmv_accel_Pipeline_init_loop	13	57	0	0	0
spmv_accel_1	spmv_accel	spmv_accel_Pipeline_compute_loop	4607	4359	0	0	0
spmv_accel_1	spmv_accel	spmv_accel_Pipeline_wb_loop	567	195	0	0	0
spmv_accel_1	spmv_accel	spmv_accel	30716	50478	0	16	0

```

changelog: T003-020-0 - ProjectAble/Final/SISAC-Benchmarks/SPM/row_size/2/complex/complex
Matrix/matrix file read successfully. M=500, N=500, NN=500
Processed for stream 1: row_size=25, values_size=32
Processed for stream 2: row_size=25, values_size=32
Processed for stream 3: row_size=25, values_size=32
Processed for stream 4: row_size=25, values_size=32
col_size=96, compact_indices_size=590
Found Platform
Platform Name: Xilinx
Reading spm_simple.xclbin
Reading spm_simple.xclbin
INFO: [HLS-BU 0] Path of the simulation directory : /home/chingmei/ProjectAble/Final/SISAC-Benchmarks/SPM/row_size/2/complex/complex
user socket name is /my/chingmei/Device/0_118165
INFO: [HLS-BU 0] Hardware emulation runs simulation underneath. Using a large data set will result in long
. The flow uses approximate models for cross reconfigurations and interconnect and hence the performance data from
configuring dataflow model with erit polling
scheduler config erit(), dataflow(1), slots(32), cdns(0), cds(0), cds(0), cds(1)
Launching SPM kernel...

=== 2-MW PARALLEL SPM PERFORMANCE ===
Matrix: ./data/large_matrix.mtx
Dimensions: 500x500, non-zeroes: 2500
Execution time: 10000 ms
Performance: 4.0000e-07 GROUPS
Memory Bandwidth: 2.3000e+06 GB/s
Result verification: PASSED
Machine error: 0
=====

SPM result (first 10 elements or all if fewer):
y[0] = 1.3
y[1] = 1.8
y[2] = 2.7
y[3] = 1.7
y[4] = 6.9
y[5] = 2
y[6] = 1.3
y[7] = 1.1
y[8] = 2.1
y[9] = 1.5

```

HiSparse: High-Performance Sparse Linear Algebra on HBM-Equipped FPGAs

- Challenges desired to solve
 - Timing closure on a multi-die FPGA
 - Irregularity in the compute pattern of SpMV causes bank conflicts and carried data dependence
 - Multi-cycle for floating-point number in addition stage complicates the case of RAW hazard
- Proposed solutions
 - Split kernel + relay unit
 - load-store forwarding mechanism
 - Row interleaving + stall unit

HiSparse: Pipelined PE with Load-Store Forwarding

- In-flight write queue (IFWQ)
- Dependence resolution logic

Operation \ Cycle	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Input a , addr = 0		MUL		ADD															
Input c , addr = 1			Bubbles		RD & WR														
Input c , addr = 1						Bubbles		MUL		ADD									
Input d , addr = 1									MUL		ADD								
Deferred Input b , addr = 0										Bubbles		RD & WR							
Deferred Input b , addr = 0													Bubbles		MUL		ADD		

(a) Using registers.

Operation \ Cycle	0	1	2	3	4	5	6	7	8	9	IFWQ Entries	0	1	2	3	4
Input a , addr = 0		RD & MUL		ADD	WR						Cycle 0, 1, 2					
Input c , addr = 1			RD & MUL		ADD	WR					Cycle 3	a				
Input d , addr = 1				RD & MUL		ADD	WR				Cycle 4	c	a			
Deferred Input b , addr = 0					RD & MUL		ADD	WR			Cycle 5	d	c	a		

(b) Using load-store forwarding — Red arrows indicate the RAW dependencies. Blue arrows indicate the data forwarding to resolve dependencies.

HiSparse: Pipelined PE with Load-Store Forwarding

- Stage 1: Get address from the payload
- Stage 2: Check the RAW hazard
- Stage 3: Update the IFWQ
- Usable scenario: when number of cycle of the addition logic is equal to one

```
1 while (!exit) {
2   #pragma HLS pipeline
3   #pragma HLS dependence variable=out_buffer inter RAW false
4   // fetch input and get bank address
5   pld = in.read();
6   addr = get_addr(pld.row_idx);
7   // multiplication and read
8   update = pld.mat_value * pld.vec_value;
9   mem_value = out_buffer[addr];
10  // dependence resolution logic
11  fwd_value = 0;
12  has_RAW = false;
13  for (int i = 0; i < IFWQ_DEPTH; i++) {
14    #pragma HLS unroll
15    if (addr == IFWQ[i].addr && IFWQ[i].valid) {
16      has_RAW = true;
17      fwd_value = IFWQ[i].data;
18      break;
19    }
20  }
21  base = has_RAW ? fwd_value : mem_value;
22  // addition and write
23  new_value = base + update;
24  out_buffer[addr] = new_value;
25  // update IFWQ
26  // IFWQ[0] stores the latest in flight write
27  for (i = IFWQ_DEPTH - 1; i > 0; i--) {
28    #pragma HLS unroll
29    IFWQ[i] = IFWQ[i - 1];
30  }
31  IFWQ[0].addr = addr;
32  IFWQ[0].data = new_value;
33  IFWQ[0].valid = true;
34 }
```

HiSparse: Floating-Point Implementation

- Inter-iteration carried dependencies
- Naive way: duplicate the output buffer to multiple partial buffer
- Suggested way: Row interleaving

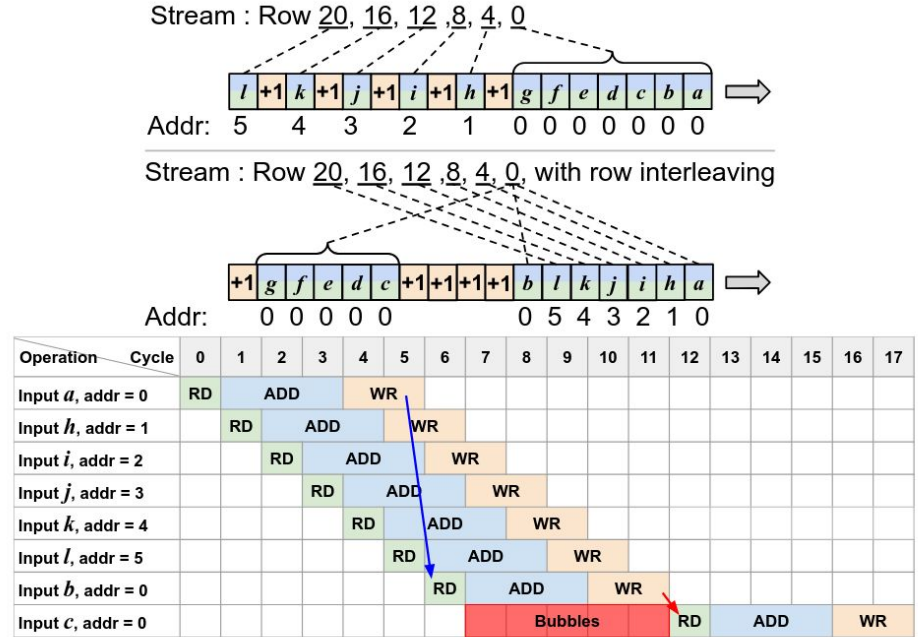


Figure 10: PE with row interleaving — Blue and red arrows indicate the dependencies resolved by row interleaving and stalling, respectively. "Addr" indicates the output buffer bank address.

HiSparse: Timing Closure on Multi-Die FPGAS

- Concepts of die and SLR (Super logic regions)
- Split kernel vs monolithic kernel
- Relay Unit
- `create_pblock <k0>; resize_pblock ... -add_slrs {SLR0}`

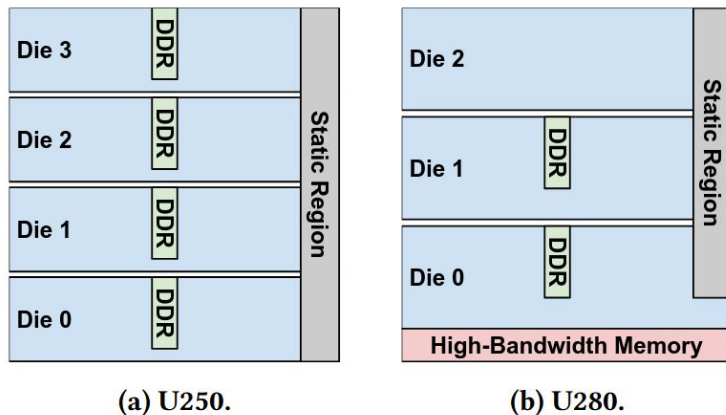


Figure 1: Comparison between Xilinx Alveo U250 and U280.

```
#include <hls_stream.h>
#include <ap_int.h>

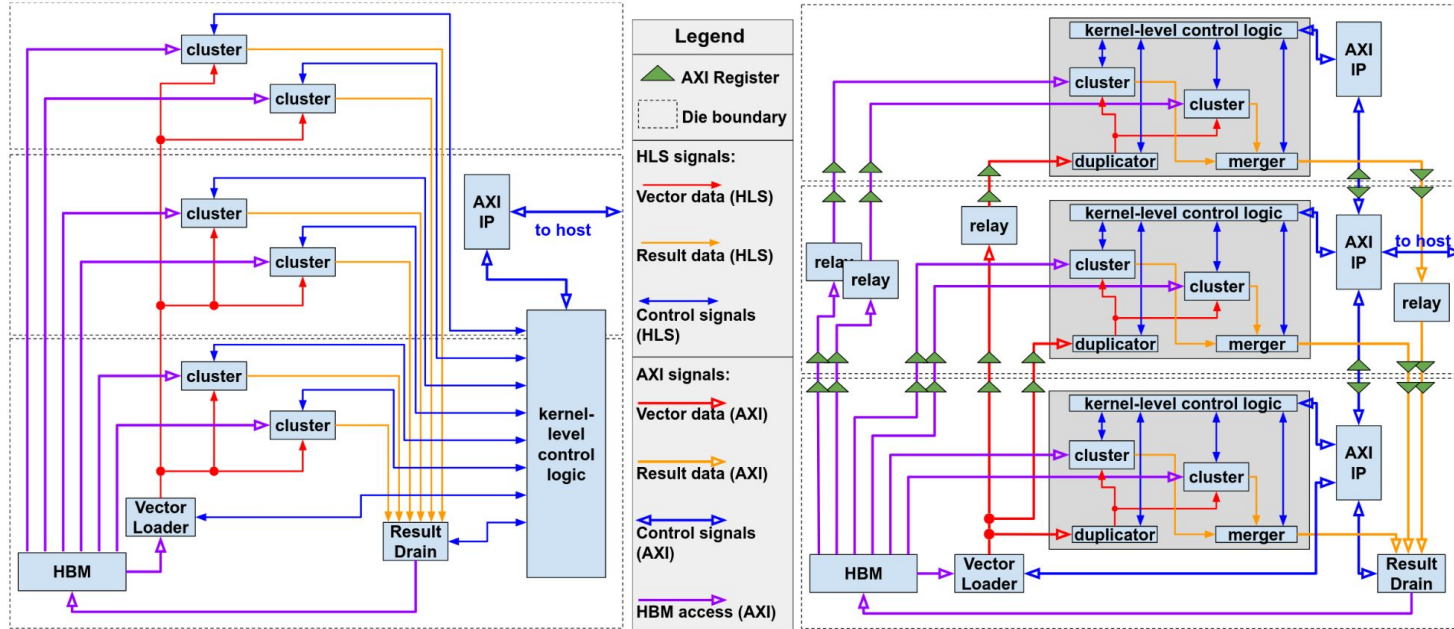
#include "common.h"

extern "C" {
void k2k_relay(
    hls::stream<VEC_AXIS_T> &in,
    hls::stream<VEC_AXIS_T> &out
) {
    #pragma HLS interface ap_ctrl_none port=return

    #pragma HLS interface axis register both port=in
    #pragma HLS interface axis register both port=out

#ifdef __SYNTHESIS__
    bool exit = false;
    while (!exit) {
        VEC_AXIS_T pkt = in.read();
        out.write(pkt);
        exit = (pkt.user == EOS);
    }
#else
    while (1) {
        #pragma HLS pipeline II=1
        VEC_AXIS_T pkt = in.read();
        out.write(pkt);
    }
#endif
} // kernel
} // extern "C"
```

HiSparse: Floorplanning of HiSparse



(a) Monolithic SpMV — inter-die signals degrade timing.

(b) Split-kernel SpMV — inter-die signals are pipelined.

Reports for Hardware emulation and Synthesis

```

|DSP| - | - | - | - | - |
|Expression| - | - | 0 | 44 | - |
|FIFO| - | - | 108 | 12528 | - |
|Instance| 0 | 210 | 200763 | 366578 | 192 |
|Memory| - | - | - | - | - |
|Multiplexer| - | - | - | 63 | - |
|Register| - | - | 13 | - | - |
+-----+-----+-----+-----+-----+
|Total| 0 | 210 | 200884 | 379213 | 192 |
+-----+-----+-----+-----+-----+
|Available SLR| 1344 | 3008 | 869120 | 434560 | 320 |
+-----+-----+-----+-----+-----+
|Utilization SLR (%)| 0 | 6 | 23 | 87 | 60 |
+-----+-----+-----+-----+-----+
|Available| 4032 | 9024 | 2607360 | 1303680 | 960 |
+-----+-----+-----+-----+-----+
|Utilization (%)| 0 | 2 | 7 | 29 | 20 |
+-----+-----+-----+-----+-----+

+ Detail:
reports/spmv_sk2/hls_reports/spmv_sk2_csynth.rpt 77.1 12%

+-----+-----+-----+-----+-----+
|Name| BRAM_18K| DSP | FF | LUT | URAM|
+-----+-----+-----+-----+-----+
|DSP| - | - | - | - | - |
|Expression| - | - | 0 | 44 | - |
|FIFO| - | - | 108 | 12528 | - |
|Instance| 0 | 210 | 200763 | 366578 | 192 |
|Memory| - | - | - | - | - |
|Multiplexer| - | - | - | 63 | - |
|Register| - | - | 13 | - | - |
+-----+-----+-----+-----+-----+
|Total| 0 | 210 | 200884 | 379213 | 192 |
+-----+-----+-----+-----+-----+
|Available SLR| 1344 | 3008 | 869120 | 434560 | 320 |
+-----+-----+-----+-----+-----+
|Utilization SLR (%)| 0 | 6 | 23 | 87 | 60 |
+-----+-----+-----+-----+-----+
|Available| 4032 | 9024 | 2607360 | 1303680 | 960 |
+-----+-----+-----+-----+-----+
|Utilization (%)| 0 | 2 | 7 | 29 | 20 |
+-----+-----+-----+-----+-----+
reports/spmv_sk1/hls_reports/spmv_sk1_csynth.rpt 74.1 11%

+-----+-----+-----+-----+-----+
|Name| BRAM_18K| DSP | FF | LUT | URAM|
+-----+-----+-----+-----+-----+
|DSP| - | - | - | - | - |
|Expression| - | - | 0 | 44 | - |
|FIFO| - | - | 108 | 12528 | - |
|Instance| 0 | 210 | 200763 | 366578 | 192 |
|Memory| - | - | - | - | - |
|Multiplexer| - | - | - | 63 | - |
|Register| - | - | 13 | - | - |
+-----+-----+-----+-----+-----+
|Total| 0 | 210 | 200884 | 379213 | 192 |
+-----+-----+-----+-----+-----+
|Available SLR| 1344 | 3008 | 869120 | 434560 | 320 |
+-----+-----+-----+-----+-----+
|Utilization SLR (%)| 0 | 6 | 23 | 87 | 60 |
+-----+-----+-----+-----+-----+
|Available| 4032 | 9024 | 2607360 | 1303680 | 960 |
+-----+-----+-----+-----+-----+
|Utilization (%)| 0 | 2 | 7 | 29 | 20 |
+-----+-----+-----+-----+-----+
reports/spmv_sk0/hls_reports/spmv_sk0_csynth.rpt 51.0-1

```